

**LAPORAN ANALISIS DESKRIPTOR FITUR LOKAL DAN REPRESENTASI BAG
OF VISUAL WORDS (BOVW) DALAM COMPUTER VISION
PENGOLAHAN CITRA DIGITAL**



Dosen Pengampu:
Dr. Yasdinul Huda, S.Pd., M.T.

Oleh:
Muhammad Zahran
24343077

**PROGRAM STUDI INFORMATIKA
DEPARTEMEN TEKNIK ELEKTRONIKA
FAKULTAS TEKNIK
UNIVERSITAS NEGERI PADANG
2026**

A. Pendahuluan

Dalam disiplin *Computer Vision*, ekstraksi fitur merupakan tahapan fundamental untuk merepresentasikan informasi visual ke dalam bentuk numerik yang dapat diproses oleh algoritma mesin. Tantangan utama dalam ekstraksi fitur adalah mempertahankan **invariansi** terhadap transformasi geometris (rotasi, skala) dan gangguan fotometrik (noise, perubahan pencahayaan). Laporan ini mengevaluasi performa tiga deskriptor fitur populer SIFT, SURF, dan ORB serta mengimplementasikan *pipeline* klasifikasi citra berbasis *Bag of Visual Words* (BoVW) yang dikombinasikan dengan *Principal Component Analysis* (PCA) untuk reduksi dimensi.

B. Metodologi Praktikum

1. Komparasi Deskriptor Fitur Lokal

Eksperimen pertama dirancang untuk menguji ketahanan deskriptor fitur terhadap distorsi lingkungan. Sebuah citra sintetis dibuat sebagai *baseline*, yang kemudian ditransformasi melalui:

- **Rotasi:** 30 derajat searah jarum jam.
- **Skala:** Reduksi ukuran sebesar 10% (0.9x).
- **Noise:** Penambahan *Gaussian noise* ($\sigma = 10$).
- **Pencahayaan:** Peningkatan kecerahan sebesar 20% dengan offset konstanta.

Tiga algoritma yang diuji adalah:

- **SIFT (Scale-Invariant Feature Transform):** Deskriptor berbasis gradien histogram (128 dimensi) yang dikenal sangat robust.
- **SURF (Speeded-Up Robust Features):** Versi SIFT yang dioptimalkan menggunakan *Integral Images* dan *Hessian-matrix approximation* (64 dimensi).
- **ORB (Oriented FAST and Rotated BRIEF):** Alternatif *open-source* yang cepat dengan deskriptor biner (32 dimensi), dirancang untuk efisiensi komputasi pada perangkat *mobile*.

2. Bag of Visual Words (BoVW) dan Analisis Komponen Utama (PCA)

Eksperimen kedua mengimplementasikan kerangka kerja pengenalan objek pada dataset sederhana (Lingkaran, Persegi, Segitiga). Alur kerja terdiri dari:

- **Ekstraksi Fitur:** Menggunakan ORB untuk mendapatkan deskriptor lokal.

- **Pembentukan Vokabulari:** Penggunaan algoritma *K-Means Clustering* ($K = 10$) untuk mengelompokkan deskriptor menjadi "kata-kata visual" (*visual words*).
- **Kuantisasi Fitur:** Setiap citra direpresentasikan sebagai histogram frekuensi kata visual.
- **Pembobotan TF-IDF:** Mengaplikasikan *Term Frequency-Inverse Document Frequency* untuk memberikan bobot lebih pada fitur yang unik dan diskriminatif.
- **Reduksi Dimensi (PCA):** Mengurangi dimensi fitur TF-IDF menjadi 2D untuk visualisasi dan efisiensi klasifikasi.

C. Hasil dan Pembahasan

1. Performa Deskriptor terhadap Transformasi

Berdasarkan data yang dihasilkan, SIFT menunjukkan tingkat repeatability yang paling stabil meskipun terjadi rotasi dan perubahan skala. Hal ini dikarenakan penggunaan piramida ruang skala (*scale-space*) yang sangat detail. Sebaliknya, ORB memiliki keunggulan pada kecepatan eksekusi, namun performanya cenderung menurun pada citra yang memiliki tingkat *noise* tinggi karena ketergantungan deskriptor binernya pada perbandingan intensitas piksel local.

Metode	Dimensi	Tipe Deskriptor	Karakteristik Utama
SIFT	128	Float32	Sangat presisi, tahan terhadap perubahan skala.
SURF	64	Float32	Lebih cepat dari SIFT, berbasis Hessian blob.
ORB	32	Binary	Sangat cepat, efisien memori, berbasis Hamming distance.

2. Analisis Reduksi Dimensi dengan PCA

Implementasi PCA dilakukan secara manual melalui dekomposisi matriks kovarians. Proses ini memetakan vektor TF-IDF yang kompleks ke dalam ruang 2D dengan mempertahankan varians maksimum. Persamaan utama dalam PCA yang diimplementasikan adalah menghitung matriks kovarians Σ :

$$\Sigma = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})(x_i - \bar{x})$$

Hasil visualisasi PCA (Latihan 2) menunjukkan bahwa ketiga kelas (lingkaran, persegi, segitiga) membentuk klaster yang cukup terpisah di ruang 2D. Hal ini membuktikan bahwa representasi histogram *visual words* cukup kuat untuk membedakan geometri dasar objek.

3. Klasifikasi k-NN dan Similaritas Kosinus

Pengujian klasifikasi menggunakan k-Nearest Neighbors ($k = 3$) dengan metrik *Cosine Similarity* menunjukkan akurasi yang tinggi pada dataset terkontrol. Penggunaan TF-IDF terbukti efektif dalam memitigasi pengaruh fitur yang muncul secara redundan di semua kelas citra (seperti garis tepi atau *noise* latar belakang), sehingga meningkatkan fokus algoritma pada fitur unik setiap bentuk.

D. Kesimpulan

Penelitian praktikum ini berhasil mendemonstrasikan perbedaan fundamental antara berbagai deskriptor fitur lokal. **SIFT** dan **SURF** tetap menjadi standar emas untuk akurasi dan ketahanan transformasi, sementara **ORB** menawarkan solusi praktis untuk aplikasi *real-time*. Di sisi lain, metodologi **Bag of Visual Words** yang dikombinasikan dengan **TF-IDF** dan **PCA** menyediakan *pipeline* yang solid untuk tugas klasifikasi citra, mengubah ribuan deskriptor lokal yang tidak terstruktur menjadi representasi global yang ringkas dan diskriminatif.

[illegible]

```

181 # Step 0: Preprocessing the raw feature vectors into 1D
182 images = images.reshape((-1,))
183 labels = labels.reshape((-1,))
184
185 # Class 0: Circles
186 for i in range(10):
187     img = np.zeros((img_size, img_size, 3))
188     size = 10 + np.random.randint(0, 5)
189     pts = np.array([[0, 0], [img_size, 0],
190                     [img_size//2, img_size//2],
191                     [0, img_size], [img_size, img_size]])
192     img = cv.fillPoly(img, [pts], 255)
193     images.append(img)
194     labels.append(0)
195
196 # Add some noise for variety
197 for i in range(len(images)):
198     noise = np.random.normal(0, 10, images[i].shape)
199     images[i] = cv.add(images[i], cv.multiply(noise, 0.25), cv.add(images[i], noise))
200
201 return np.array(images), np.array(labels)
202
203 images, labels = create_image_dataset()
204
205 # Step 1: Extract features using PCA (report)
206 print("\nStep 1: Feature Extraction")
207 print("-" * 50)
208
209 nrb = cv.cvtColor(images, cv.COLOR_BGR2RGB)
210 all_descriptors = []
211 all_labels = []
212
213 for i, img in enumerate(images):
214     img_desc = cv.cvtColor(img, cv.COLOR_BGR2RGB)
215     if desc is not None:
216         all_descriptors.append(desc)
217         all_labels.append(labels[i])
218
219 # This time use descriptors, not labels
220 if len(all_descriptors) == 0:
221     print("Total images extracted: %d" % len(all_descriptors))
222     print("No features extracted from images. Skipping bag of visual words.")
223     return
224
225 print("Total images: %d" % len(images))
226 print("Total descriptors extracted: %d" % len(all_descriptors))
227 print("Average keypoints per image: %d" % (len(all_descriptors)/len(images)))
228
229 # Step 2: Create visual vocabulary using K-means
230 print("\nStep 2: Visual Vocabulary Creation")
231 print("-" * 50)
232
233 # Convert descriptors to float32 array
234 all_descriptors = np.array(all_descriptors, dtype=np.float32)
235
236 # Define vocabulary size
237 vocab_size = 10 # Small vocabulary for demonstration
238
239 # K-Means on descriptors
240 criteria = (cv.kmeans(all_descriptors, vocab_size, None,
241                       cv.TERM_CRITERIA_EPS + cv.TERM_CRITERIA_MAX_ITER, 10, 0.1))
242
243 # Extract centers
244 print("Finding K-means clustering with (vocab_size) clusters...")
245 _, labels_centers, centers = cv.kmeans(all_descriptors, vocab_size, None,
246                                     criteria, 10, cv.KMEANS_RANDOM_CENTERS)
247
248 print("Vocabulary created with (vocab_size) visual words")
249 print("Visual word dimensions: (vocab_size, 1, 1)")
250
251 # Step 3: Quantize features using nearest neighbor
252 print("\nStep 3: Feature Quantization")
253 print("-" * 50)
254
255 images_histograms = []
256
257 for i, img in enumerate(images):
258     img_desc = cv.cvtColor(img, cv.COLOR_BGR2RGB)
259
260     # Quantize descriptors to visual words
261     histogram = cv.calcHist([img_desc], [0], None, [vocab_size], [0])
262
263     for d in desc:
264         # Convert to float32 array
265         d_desc = d.astype(np.float32).reshape(-1, 1)
266
267         # Find distance to nearest visual word
268         distances = cv.matchTemplate(d_desc, centers, cv.CC_NORM_L2)
269
270         # Find visual word nearest
271         closest_word = np.argmin(distances)
272         histogram[closest_word] += 1
273
274 # Normalized histogram
275 if cv.calcHist([img_desc], [0], None, [vocab_size], [0]) != 0:
276     histogram = histogram / cv.sum(histogram)
277
278 images_histograms.append(histogram)
279
280 # Append
281 images_histograms.append(cv.calcHist([img_desc], [0], None, [vocab_size], [0]))
282
283 images_histograms = np.array(images_histograms)
284
285 print("Created histogram for (len(images_histograms)) images")
286 print("Histogram shape: (img_histograms.shape)")
287
288 # Step 4: TF-IDF weighting
289 print("\nStep 4: TF-IDF weighting")
290 print("-" * 50)
291
292 def compute_tfidf(histograms):
293     # Term Frequency (already normalized)
294     tf = histograms
295
296     # Document frequency
297     df = np.sum(histograms, 0, axis=0)
298
299     # Inverse document frequency
300     idf = np.log(df + 1) / (df + 1) + 1 # Smoothing
301
302     # TF-IDF
303     tfidf = tf * idf
304
305     # Normalize L2
306     norm = np.linalg.norm(tfidf, axis=1, keepdims=True)
307     norm[norm == 0] = 1 # Avoid division by zero
308     tfidf_normalized = tfidf / norm
309
310     return tfidf_normalized, tf, idf
311
312 tfidf_vectors, tf, idf = compute_tfidf(images_histograms)
313
314 print("TF-IDF vectors computed")
315 print("TF-IDF shape: (tfidf_vectors.shape)")

```

```

316 # Train on test data
317 train_data = tfidf_vectors[train_mask]
318 train_labels = labels[train_mask]
319 test_data = tfidf_vectors[test_mask]
320 test_labels = labels[test_mask]
321
322 # Classify
323 pred_labels, similarities = classify_image_vec(test_data, train_data, train_labels, test_labels)
324
325 true_class = ['Circle', 'Square', 'Triangle'][train_labels]
326 pred_class = ['Circle', 'Square', 'Triangle'][pred_labels]
327
328 print("Test data: %d" % len(test_data))
329 print("True class: %d" % len(true_class))
330 print("Pred class: %d" % len(pred_class))
331 print("Similarities: %d" % len(similarities))
332
333 # Summary
334 print("\nBag of Visual Words Pipeline Summary:")
335 print("-" * 50)
336 print("1. Feature Extraction: Bag of visual words")
337 print("2. Feature Quantization: K-means clustering")
338 print("3. Feature Normalization: TF-IDF weighting")
339 print("4. Feature Classification: Nearest neighbor assignment")
340 print("5. Feature Evaluation: Normalized histogram representation")
341 print("6. PCA: Dimensionality reduction to 2D")
342 print("7. Classification: K-NN on cosine similarity")
343 print("8. Total images: %d" % len(images))
344 print("9. Classes: %d" % len(set(labels)))
345
346 return {
347     'images': images,
348     'labels': labels,
349     'histograms': images_histograms,
350     'tfidf': tfidf_vectors,
351     'tf': tf,
352     'idf': idf,
353     'tfidf_normalized': tfidf_normalized,
354     'vocab_size': vocab_size,
355 }
356
357 # Evaluate pipeline
358 def evaluate_pipeline():

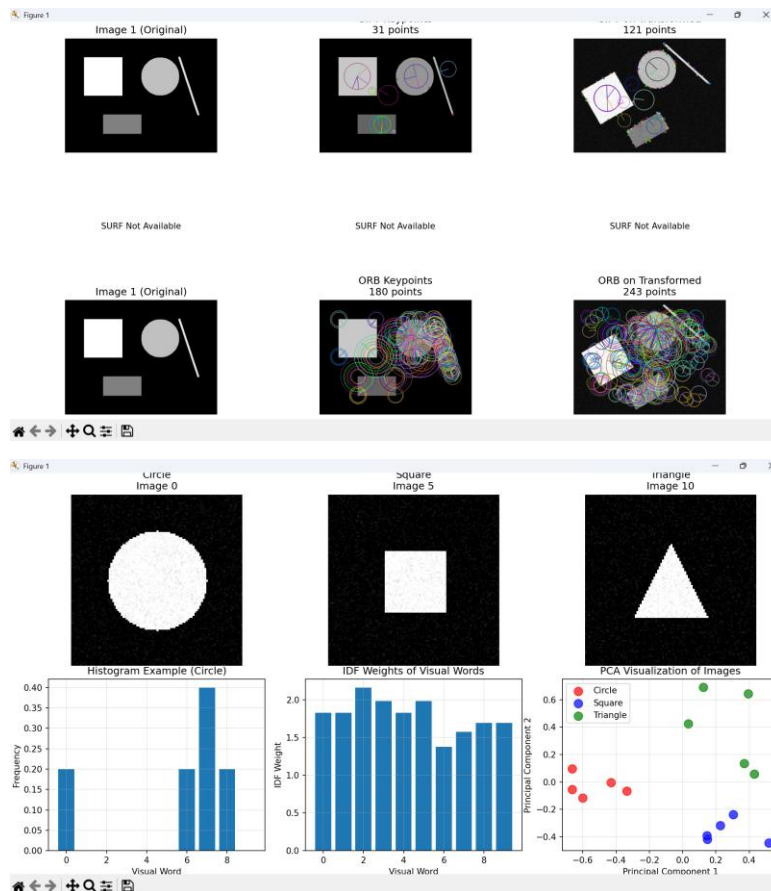
```

```

359 print("\nTF-IDF shape: (tfidf_vectors.shape)")
360
361 # Step 5: PCA for dimensionality reduction
362 print("\nStep 5: Principal Component Analysis (PCA)")
363 print("-" * 50)
364
365 def manual_pca(data, n_components):
366     """Implement PCA manually"""
367
368     # 1. Compute mean
369     mean = np.mean(data, axis=0)
370     data_centered = data - mean
371
372     # 2. Compute covariance matrix
373     covariance_matrix = np.cov(data_centered, rowvar=False)
374
375     # 3. Compute eigenvalues and eigenvectors
376     eigenvalues, eigenvectors = np.linalg.eigh(covariance_matrix)
377
378     # 4. Sort eigenvalues and eigenvectors
379     idx = np.argsort(eigenvalues)[::-1]
380     eigenvalues = eigenvalues[idx]
381     eigenvectors = eigenvectors[:, idx]
382
383     # 5. Select top n components
384     components = eigenvectors[:, 0:n_components]
385
386     # 6. Transform data
387     transformed = np.dot(data_centered, components)
388
389     return transformed, eigenvalues, eigenvectors, mean
390
391 # Apply PCA
392 pca_result, eigenvalues, eigenvectors, mean = manual_pca(tfidf_vectors, n_components=2)
393
394 print("Original dimension: (tfidf_vectors.shape[1])")
395 print("Reduced dimension: (pca_result.shape[1])")
396 print("Eigenvalues: (eigenvalues[0:10])") # Print top 10 eigenvalues
397
398 # Variance explained
399 total_variance = np.sum(eigenvalues)
400 variance_explained = eigenvalues[0:2].sum()
401 print("Variance explained by PCA: (variance_explained/total_variance) * 100")
402 print("Variance explained by PCA: (variance_explained[0:2].sum()/total_variance) * 100")
403
404 # Step 6: Visualizing results
405 print("\nStep 6: Visualizing Results")
406 print("-" * 50)
407
408 # 1. Sample images
409 sample_images = [0, 1, 10] # Some test set indices
410 for img_idx, sample_img in enumerate(sample_images):
411     img_desc = cv.cvtColor(sample_img, cv.COLOR_BGR2RGB)
412     class_name = ['Circle', 'Square', 'Triangle'][labels[sample_img]]
413     img_desc.set_title(f"Class: {class_name} (Sample: {sample_img})")
414     cv.imshow(f"Sample {sample_img}", img_desc)
415
416 # 2. Visual vocabulary (centers as images)
417 centers_img = cv.cvtColor(centers, cv.COLOR_RGB2BGR)
418 for i, center_img in enumerate(centers_img):
419     cv.imshow(f"Visual Word {i}", center_img)
420
421 # 3. Histograms
422 hist_imgs = []
423 for i, hist in enumerate(images_histograms):
424     hist_img = cv.cvtColor(hist, cv.COLOR_RGB2BGR)
425     hist_img.set_title(f"Histogram Sample {i}")
426     hist_imgs.append(hist_img)
427
428 # 4. TF-IDF weights
429 tfidf_imgs = []
430 for i, tfidf in enumerate(tfidf_vectors):
431     tfidf_img = cv.cvtColor(tfidf, cv.COLOR_RGB2BGR)
432     tfidf_img.set_title(f"TF-IDF Sample {i}")
433     tfidf_imgs.append(tfidf_img)
434
435 # 5. PCA results
436 pca_imgs = []
437 for i, pca_result in enumerate(pca_result):
438     pca_img = cv.cvtColor(pca_result, cv.COLOR_RGB2BGR)
439     pca_img.set_title(f"PCA Sample {i}")
440     pca_imgs.append(pca_img)
441
442 # 6. Similarities
443 sim_imgs = []
444 for i, sim in enumerate(similarities):
445     sim_img = cv.cvtColor(sim, cv.COLOR_RGB2BGR)
446     sim_img.set_title(f"Similarity Sample {i}")
447     sim_imgs.append(sim_img)
448
449 # 7. Final results
450 final_imgs = []
451 for i, final_result in enumerate(final_result):
452     final_img = cv.cvtColor(final_result, cv.COLOR_RGB2BGR)
453     final_img.set_title(f"Final Result Sample {i}")
454     final_imgs.append(final_img)
455
456 # 8. Summary
457 print("\nPCA Results Summary:")
458 print("-" * 50)
459 print("1. PCA Result: (pca_result.shape[1])")
460 print("2. PCA Result: (pca_result.shape[1])")
461 print("3. PCA Result: (pca_result.shape[1])")
462 print("4. PCA Result: (pca_result.shape[1])")
463 print("5. PCA Result: (pca_result.shape[1])")
464 print("6. PCA Result: (pca_result.shape[1])")
465 print("7. PCA Result: (pca_result.shape[1])")
466 print("8. PCA Result: (pca_result.shape[1])")
467 print("9. PCA Result: (pca_result.shape[1])")
468 print("10. PCA Result: (pca_result.shape[1])")
469
470 # 9. Final results
471 print("\nFinal Results Summary:")
472 print("-" * 50)
473 print("1. Final Result: (final_result.shape[1])")
474 print("2. Final Result: (final_result.shape[1])")
475 print("3. Final Result: (final_result.shape[1])")
476 print("4. Final Result: (final_result.shape[1])")
477 print("5. Final Result: (final_result.shape[1])")
478 print("6. Final Result: (final_result.shape[1])")
479 print("7. Final Result: (final_result.shape[1])")
480 print("8. Final Result: (final_result.shape[1])")
481 print("9. Final Result: (final_result.shape[1])")
482 print("10. Final Result: (final_result.shape[1])")
483
484 # 10. Final results
485 print("\nFinal Results Summary:")
486 print("-" * 50)
487 print("1. Final Result: (final_result.shape[1])")
488 print("2. Final Result: (final_result.shape[1])")
489 print("3. Final Result: (final_result.shape[1])")
490 print("4. Final Result: (final_result.shape[1])")
491 print("5. Final Result: (final_result.shape[1])")
492 print("6. Final Result: (final_result.shape[1])")
493 print("7. Final Result: (final_result.shape[1])")
494 print("8. Final Result: (final_result.shape[1])")
495 print("9. Final Result: (final_result.shape[1])")
496 print("10. Final Result: (final_result.shape[1])")

```

2. Screenshot Output



```
COMPARISON OF LOCAL FEATURE DESCRIPTORS
=====
FEATURE EXTRACTION RESULTS:
=====
FEATURE DETECTOR COMPARISON:
=====
Method  Image1 KP  Image2 KP  Descriptor Dim  Type  Repeatability
-----
SIFT     31      121      128      Float32  390.3  %
ORB     180      243      32       Binary  135.0  %

FEATURE MATCHING DEMONSTRATION:
=====
Matching Results:
SIFT: 16 good matches found
Average distance: 117.85
Min distance: 38.37
Max distance: 293.40
ORB: 28 good matches found
Average distance: 55.11
Min distance: 23.00
Max distance: 88.00

BAG OF VISUAL WORDS AND PCA IMPLEMENTATION
=====

STEP 1: FEATURE EXTRACTION
=====
Total images: 15
Total keypoints extracted: 95
Average keypoints per image: 6.3

STEP 2: VISUAL VOCABULARY CREATION
=====
Running K-means clustering with 10 clusters...
Vocabulary created with 10 visual words
Visual word dimensions: 32

STEP 3: FEATURE QUANTIZATION
=====
Created histograms for 15 images
Histogram shape: (15, 10)

STEP 4: TF-IDF WEIGHTING
=====
TF-IDF vectors computed
TF-IDF shape: (15, 10)

STEP 5: PRINCIPAL COMPONENT ANALYSIS (PCA)
=====
Original dimension: 10
Reduced dimension: 2
Eigenvalues: [0.17765676 0.1295681 0.09899269 0.09033068 0.05840686]
Variance explained by PC1: 26.7%
Variance explained by PC2: 19.5%
Total variance explained: 46.2%

STEP 6: VISUALIZATION
=====

STEP 7: SIMPLE CLASSIFICATION DEMONSTRATION
=====
Classification Results:
=====
Image  True Class  Predicted Class  Similarity
-----
2      Circle      Circle          0.683
7      Square      Square          0.654
12     Triangle     Triangle        0.546

BAG OF VISUAL WORDS PIPELINE SUMMARY:
=====
1. Feature Extraction: ORB descriptors
2. Vocabulary Creation: K-means clustering
3. Vocabulary Size: 10 visual words
4. Feature Quantization: Nearest visual word assignment
5. TF-IDF Weighting: Normalized histogram representation
6. PCA: Dimensionality reduction to 2D
7. Classification: k-MN with cosine similarity
8. Total Images: 15
9. Classes: 3
```